

```
In [ ]: import marimo as mo
import random
from ortools.sat.python import cp_model
```

Sudoku genérico como CSP

Organização do notebook

Durante a realização deste projeto tentamos seguir os nomes sugeridos no enunciado sempre que possível. Correspondência com os requisitos:

Requisito	No notebook
R1 – grupo genérico	classe <code>box</code> (<code>add</code> , <code>matriz</code> e o auxiliar <code>fixas</code>)
R2 – bloco $n \times n$	classe <code>cube(i, j, n)</code>
R3 – troço reto	classe <code>path(inicio, fim, n)</code>
R4 – pistas aleatórias	função <code>pistas_random(n, k, seed)</code>
R5 – modelo CSP	classe <code>Modelo</code> (<code>add</code> , <code>solve</code>)
R6 – Sudoku completo	<code>montar_sudoku</code> e <code>resolver_sudoku</code>
Validação	<code>validar_grelha</code> , <code>validar_pistas</code> , <code>verificar_add</code>
Bónus	Sudoku diagonal (X-Sudoku) e escala até $n = 6$

Nenhuma função tem o tamanho 9×9 fixo: tudo é calculado a partir de n (ou de $N = n^2$).

Grupos de células: `box` , `cube` e `path`

Estrutura de dados. O `box` guarda as células num dicionário `(linha, coluna) → valor ou None` . Escolhemos o dicionário porque corresponde diretamente ao que o enunciado descreve e porque impede que a mesma célula apareça duas vezes no grupo. Isso seria um problema: uma variável repetida dentro de um "todos diferentes" tornava o modelo impossível.

`add` . Valida tudo antes de alterar o grupo: as coordenadas têm de ser inteiras e estar dentro da grelha, e o valor tem de ser inteiro em $[1, n^2]$. Caso contrário levanta exceção (`TypeError` , `IndexError` ou `ValueError`). Também não deixa que uma célula já fixa passe a ter outro valor.

`matriz()` devolve a grelha $n^2 \times n^2$ com o valor nas células fixas e 0 nas restantes.

Generalidade. O `box` só conhece `n`, porque precisa dele para validar coordenadas e valores. Não sabe nada de linhas, colunas ou blocos, e por isso serve também para as pistas e para as diagonais.

cube e path só decidem que células entram no grupo e usam o `add` do `box`, por isso herdam as mesmas validações:

- `cube(i, j)` começa no canto $(i \cdot n, j \cdot n)$ e adiciona as $n \times n$ células do bloco;
- `path(inicio, fim)` aceita troços horizontais ou verticais e usa um passo de $+1$ ou -1 , para funcionar nos dois sentidos. Se o troço não for reto, dá erro.

In []: `class box:`

```
def __init__(self, celulas=None, n=3):
    # verificar se o n é valido
    if type(n) != int:
        raise TypeError("n tem de ser um inteiro")
    if n < 1:
        raise ValueError("n tem de ser maior que 0")

    self.n = n
    self.N = n * n # tamanho da grelha (ex: 9 para n=3)

    # dicionario com (linha, coluna) -> valor ou None
    self.celulas = {}

    # se deram celulas iniciais, adicionar uma a uma
    if celulas != None:
        for posicao in celulas:
            l = posicao[0]
            c = posicao[1]
            valor = celulas[posicao]
            self.add(l, c, valor)

def add(self, l, c, valor=None):
    # as coordenadas tem de ser inteiros
    if type(l) != int or type(c) != int:
        raise TypeError("as coordenadas tem de ser inteiros")

    # as coordenadas tem de estar dentro da grelha
    if l < 0 or l >= self.N or c < 0 or c >= self.N:
        raise IndexError("a célula (" + str(l) + ", " + str(c) + ")
esta fora da grelha")

    # se houver valor, tem de ser inteiro entre 1 e N
    if valor != None:
        if type(valor) != int:
            raise TypeError("o valor tem de ser um inteiro")
        if valor < 1 or valor > self.N:
            raise ValueError("o valor " + str(valor) + " tem de estar
entre 1 e " + str(self.N))

    # se a célula ja existir
```

```

        if (l, c) in self.celulas:
            valor_antigo = self.celulas[(l, c)]
            # nao deixar mudar uma celula que ja estava fixa para outro
            valor
            if valor_antigo != None and valor != None and valor_antigo !=
            valor:
                raise ValueError("a celula ja esta fixa a " +
str(valor_antigo))
                # so atualiza se o novo valor nao for None
                if valor != None:
                    self.celulas[(l, c)] = valor
            else:
                self.celulas[(l, c)] = valor

def matriz(self):
    # criar uma matriz N x N cheia de zeros
    m = []
    for i in range(self.N):
        linha = []
        for j in range(self.N):
            linha.append(0)
        m.append(linha)

    # meter os valores fixos nas posicoes certas
    for posicao in self.celulas:
        valor = self.celulas[posicao]
        if valor != None:
            l = posicao[0]
            c = posicao[1]
            m[l][c] = valor

    return m

def fixas(self):
    # devolve so as celulas que tem valor
    resultado = {}
    for posicao in self.celulas:
        if self.celulas[posicao] != None:
            resultado[posicao] = self.celulas[posicao]
    return resultado

def mostrar(self):
    print("box com n =", self.n, "e", len(self.celulas), "celulas,",
len(self.fixas()), "fixas")

```

In []: **class** cube(box):

```

def __init__(self, i, j, n=3):
    # inicializar a box vazia
    box.__init__(self, None, n)

    # os indices do bloco tem de ser inteiros
    if type(i) != int or type(j) != int:
        raise TypeError("os indices do bloco tem de ser inteiros")

    # os indices do bloco tem de estar entre 0 e n-1

```

```

        if i < 0 or i >= n or j < 0 or j >= n:
            raise IndexError("o bloco (" + str(i) + ", " + str(j) + ") nao
existe")

        # canto superior esquerdo do bloco
        linha_inicio = i * n
        coluna_inicio = j * n

        # adicionar as n x n celulas do bloco
        for l in range(linha_inicio, linha_inicio + n):
            for c in range(coluna_inicio, coluna_inicio + n):
                self.add(l, c)

```

In []: **class** path(box):

```

    def __init__(self, inicio, fim, n=3):
        # inicializar a box vazia
        box.__init__(self, None, n)

        # tirar as coordenadas do inicio e do fim
        linha0 = inicio[0]
        coluna0 = inicio[1]
        linha1 = fim[0]
        coluna1 = fim[1]

        # caso horizontal (mesma linha)
        if linha0 == linha1:
            # ver se anda para a direita ou para a esquerda
            if coluna1 >= coluna0:
                passo = 1
            else:
                passo = -1

            c = coluna0
            while c != coluna1 + passo:
                self.add(linha0, c)
                c = c + passo

        # caso vertical (mesma coluna)
        elif coluna0 == coluna1:
            # ver se anda para baixo ou para cima
            if linha1 >= linha0:
                passo = 1
            else:
                passo = -1

            l = linha0
            while l != linha1 + passo:
                self.add(l, coluna0)
                l = l + passo

        # se nao for nem horizontal nem vertical da erro
        else:
            raise ValueError("o caminho de " + str(inicio) + " para " +
str(fim) + " nao e reto")

```

Para a implementação das classes acima (box,cube,path) foi utilizado o recurso à LLM/IA "CLAUDE"

Link para a conversa : <https://claude.ai/share/c4da3290-4301-40e1-8604-4de16e72043f>

Pistas aleatórias (R4)

`pistas_random` escolhe `k` células diferentes e `k` valores diferentes (com `sample`) e devolve-os num `box` normal. Não foi precisa nenhuma classe nova.

Os valores são todos diferentes de propósito: o modelo impõe "todos diferentes" a qualquer grupo, incluindo o das pistas. Se duas pistas tivessem o mesmo valor, o puzzle ficava impossível logo à partida. Por isso `k` está limitado a n^2 .

Por omissão `k = n`, como sugere o enunciado. Usamos um gerador próprio (`random.Random(seed)`) para que a mesma seed dê sempre as mesmas pistas, o que torna os testes reproduzíveis.

```
In [ ]: def pistas_random(n, k=None, seed=None):
    N = n * n
    gerador = random.Random(seed)

    # se nao derem k pistas, usar n pistas
    if k is None:
        k = n

    # como os valores sao todos diferentes, nao dá para ter mais de N
    # pistas
    if k < 0 or k > N:
        raise ValueError("k tem de estar entre 0 e " + str(N))

    # lista com todas as celulas da grelha
    todas_celulas = []
    for i in range(N):
        for j in range(N):
            todas_celulas.append((i, j))

    # lista com todos os valores possiveis
    todos_valores = []
    for v in range(1, N + 1):
        todos_valores.append(v)

    # escolher k celulas e k valores diferentes ao calhas
    celulas = gerador.sample(todas_celulas, k)
    valores = gerador.sample(todos_valores, k)

    # criar a box com as pistas
    b = box(None, n)
    for x in range(k):
        l = celulas[x][0]
```

```

        c = celulas[x][1]
        b.add(l, c, valores[x])
    return b

```

Para a implementação da função `pistas_random` foi utilizado o recurso à LLM/IA "CLAUDE"

Link para a conversa : <https://claude.ai/share/4ab10ad3-801d-4fa8-ac1e-7ca50685c65a>

Modelo CSP (R5)

Usamos o **CP-SAT do OR-Tools**, a sugestão da disciplina. Há uma variável inteira por célula com domínio $[1, n^2]$, como pede o R5. Cada grupo dá origem a uma única restrição `AddAllDifferent`, que é exatamente a regra do enunciado.

Pensámos também num modelo `x[l][c][v]` (como na ficha 3). Também funcionava, mas precisa de $n^2 \times n^2 \times n^2$ variáveis e obriga a escrever "todos diferentes" à mão com somas. Com CP-SAT a restrição é escrita diretamente e o código fica mais curto e mais próximo do enunciado.

- `add(*grupos)` aceita qualquer número de grupos e trata-os todos da mesma forma, sem saber se vêm de um `box`, `cube`, `path` ou das pistas. Para cada um impõe `AllDifferent` e fixa as células com valor (`x == valor`). Também verifica que o grupo foi criado com o mesmo `n` que o modelo.
- `solve()` devolve a grelha como lista de listas, ou `None` se não houver solução. Como não há função objetivo, qualquer solução encontrada serve, e por isso aceitamos os estados `OPTIMAL` e `FEASIBLE`.

```

In [ ]: class Modelo:

    def __init__(self, n=3):
        self.n = n
        self.N = n * n

        # criar o modelo
        self.modelo = cp_model.CpModel()

        # uma variável inteira por célula, com valor entre 1 e N
        self.x = []
        for l in range(self.N):
            linha = []
            for c in range(self.N):
                nome = "x_" + str(l) + "_" + str(c)
                linha.append(self.modelo.NewIntVar(1, self.N, nome))
            self.x.append(linha)

    def add(self, *grupos):
        for grupo in grupos:

```

```

        # o grupo tem de ser do mesmo tamanho que o modelo
        if grupo.N != self.N:
            raise ValueError("o grupo tem N=" + str(grupo.N) + " mas o modelo tem N=" + str(self.N))

        # juntar as variaveis das celulas do grupo
        variaveis = []
        for posicao in grupo.celulas:
            l = posicao[0]
            c = posicao[1]
            variaveis.append(self.x[l][c])

        # todas as celulas do grupo tem valores diferentes
        self.modelo.AddAllDifferent(variaveis)

        # fixar as celulas que tem valor
        for posicao in grupo.celulas:
            valor = grupo.celulas[posicao]
            if valor != None:
                l = posicao[0]
                c = posicao[1]
                self.modelo.Add(self.x[l][c] == valor)

    def solve(self):
        solver = cp_model.CpSolver() #como fizemos nas aulas
        estado = solver.Solve(self.modelo)

        # se nao encontrou solucao devolve None
        if estado != cp_model.OPTIMAL and estado != cp_model.FEASIBLE:
            return None

        # construir a grelha com a solucao
        grelha = []
        for l in range(self.N):
            linha = []
            for c in range(self.N):
                linha.append(solver.Value(self.x[l][c]))
            grelha.append(linha)

        return grelha

```

Para a implementação da classe modelo foi utilizado o recurso à LLM/IA "CLAUDE"

Link para a conversa : <https://claude.ai/share/ecc0d986-1ba8-4293-a460-7ceb57e664>

Montagem e resolução (R6)

`montar_sudoku` junta ao modelo n^2 linhas e n^2 colunas (com `path`), n^2 blocos (com `cube`) e o grupo de pistas. Como as pistas são só mais um grupo, o modelo não precisa de nenhum caso especial.

Quando não há solução. Mesmo com valores diferentes, as pistas podem gerar um puzzle impossível. Exemplo em 4x4: com 1 em (0,3), 4 em (0,2), 2 em (3,0) e 3 em (1,1), a célula (0,0) não pode ter nenhum valor.

Escolhemos tentar de novo: `resolver_sudoku` gera novas pistas (seed, seed+1, ...) até 10 vezes. Se nenhuma tentativa der solução, devolve `(pistas, None)` e o notebook mostra "sem solucao".. Assim o utilizador recebe quase sempre um puzzle válido, o notebook nunca entra em ciclo infinito e a mesma seed dá sempre o mesmo resultado. Nos nossos testes isto acontece sobretudo com $n = 2$ e muitas pistas.

```
In [ ]: def montar_sudoku(n, pistas):
    N = n * n
    m = Modelo(n)

    # todas as linhas
    for l in range(N):
        m.add(path((l, 0), (l, N - 1), n))

    # todas as colunas
    for c in range(N):
        m.add(path((0, c), (N - 1, c), n))

    # todos os blocos n x n
    for i in range(n):
        for j in range(n):
            m.add(cube(i, j, n))

    # as pistas sao so mais um grupo
    m.add(pistas)
    return m

def resolver_sudoku(n, k=None, seed=None, tentativas=10):
    # se as pistas derem um puzzle impossivel, tentar outras
    for t in range(tentativas):
        if seed == None:
            s = None
        else:
            s = seed + t

        pistas = pistas_random(n, k, s)
        grelha = montar_sudoku(n, pistas).solve()

        if grelha != None:
            return pistas, grelha

    # desistir ao fim de varias tentativas
    return pistas, None
```

Para a implementação da funcao montar_sudoku e mostrar_sudoku foi utilizado o recurso à LLM/IA "CLAUDE"

Link para a conversa : <https://claude.ai/share/c71be2e6-4b94-417b-b4a5-5fbfde0bf6b8>

Apresentação

A grelha é mostrada em texto, com `|` e `-` a separar os blocos e `.` nas células vazias. É a forma mais simples que funciona para qualquer `n` e chega para verificar a solução à vista. A mesma função serve para as pistas (via `pistas.matriz()`, onde as células vazias são 0) e para a solução.

```
In [ ]: def mostrar_sudoku(grelha, n):
    if grelha == None:
        return "sem solucao"

    N = n * n
    texto = ""

    for l in range(N):
        # linha de tracos entre blocos
        if l % n == 0 and l != 0:
            texto = texto + "-" * (N * 3 + (n - 1) * 2) + "\n"

        for c in range(N):
            # barra entre blocos
            if c % n == 0 and c != 0:
                texto = texto + "| "

            valor = grelha[l][c]
            if valor == 0:
                texto = texto + " . "
            else:
                texto = texto + str(valor).rjust(2) + " "

        texto = texto + "\n"

    return texto
```

Validação automática

A validação é feita em Python simples, sem CP-SAT, para não ser o próprio solver a confirmar a sua resposta.

- `valores_corretos` verifica que uma lista tem n^2 elementos e que cada valor de 1 a n^2 aparece. Como há n^2 posições e n^2 valores obrigatórios, não pode haver repetidos.
- `validar_grelha` aplica isto a todas as linhas, colunas e blocos.
- `validar_pistas` confirma que cada pista mantém o seu valor na solução.
- `verificar_add` testa coordenadas fora da grelha e os valores 0 e $n^2 + 1$, que têm de dar erro, e alguns casos válidos, que não podem dar erro.

A célula de testes corre o fluxo completo para $n = 2$ e $n = 3$, com 5 seeds cada. Para cada n , no fim, estraga de propósito uma célula de uma grelha correta, para mostrar que o validador deteta erros e não diz sempre "OK".

```
In [ ]: def valores_corretos(valores, N):  
    # verificar que a lista tem N valores e que aparecem todos os valores  
    # de 1 a N  
  
    if len(valores) != N:  
        return False  
    for valor in range(1,N+1):  
        if valor not in valores:  
            return False  
    return True
```

```
In [ ]: def validar_grelha(grelha, n):  
    N = n * n  
    erros = []  
  
    for linha in range(N):  
        if not valores_corretos(grelha[linha], N):  
            erros.append("a linha " + str(linha) + " está errada")  
  
    for coluna in range(N):  
        colunas = []  
        for linha in range(N):  
            colunas.append(grelha[linha][coluna])  
        if not valores_corretos(colunas, N):  
            erros.append("a coluna " + str(coluna) + " está errada")  
  
    for i in range(n):  
        for j in range(n):  
            bloco = []  
            for linha in range(i * n, i * n + n):  
                for coluna in range(j * n, j * n + n):  
                    bloco.append(grelha[linha][coluna])  
            if not valores_corretos(bloco, N):  
                erros.append("o bloco (" + str(i) + ", " + str(j) + ") está  
errado")  
  
    return erros
```

```
In [ ]: def validar_pistas(grelha, pistas):  
    erros = []  
    pistas_fixas = pistas.fixas()  
  
    for pos in pistas_fixas:  
        linha = pos[0]  
        coluna = pos[1]  
  
        if grelha[linha][coluna] != pistas_fixas[pos]:  
            erros.append("a pista em " + str(pos) + " foi alterada")  
    return erros
```

```

In [ ]: def verificar_add(n):
        N = n * n
        falhas = []

        caso_mau = [
            (-1, 0, None), (N, 0, None),
            (0, -1, None), (0, N, None),
            (0, 0, 0), (0, 0, N + 1)
        ]

        for caso in caso_mau:
            b = box(None, n)
            try:
                b.add(caso[0], caso[1], caso[2])
                falhas.append("add" + str(caso) + " devia ter dado erro")
            except (IndexError, ValueError):
                pass

        caso_bom = [(0, 0, 1), (N - 1, N - 1, N), (0, N - 1, None)]

        for caso in caso_bom:
            b = box(None, n)
            try:
                b.add(caso[0], caso[1], caso[2])
            except (IndexError, ValueError):
                falhas.append("add" + str(caso) + " nao devia ter dado erro")

        return falhas

```

```

In [ ]: total_falhas = 0

for numero_teste in [2, 3]:
    print("===== n =", numero_teste, "=====")

    # o add rejeita o que deve rejeitar
    falhas_add = verificar_add(numero_teste)

    if len(falhas_add) == 0:
        print("add: OK")
    else:
        print("add: FALHOU", falhas_add)
        total_falhas = total_falhas + 1

    # fluxo completo com varias seeds
    for seed_teste in range(5):

        pistas_t, grelha_t = resolver_sudoku(numero_teste, seed=seed_teste)

        if grelha_t == None:
            print("seed", seed_teste, ": sem solucao")
            continue

        erros = validar_grelha(grelha_t, numero_teste) +
        validar_pistas(grelha_t, pistas_t)

```

```

        if len(erros) == 0:
            print("seed", seed_teste, ": ok")
        else:
            print("seed", seed_teste, ": falhou", erros)
            total_falhas = total_falhas + 1

        # o validador apanha uma grelha estragada

        pistas_t, grelha_t = resolver_sudoku(numero_teste, seed=0)

        estragada = []

        for linha in grelha_t:
            estragada.append(list(linha))
        estragada[0][0] = grelha_t[1][0]

        if len(validar_grelha(estrugada, numero_teste)) > 0:
            print("validador apanhou uma grelha errada: ok")
        else:
            print("validador apanhou uma grelha errada: falhou")
            total_falhas = total_falhas + 1

    print()
    if total_falhas == 0:
        print("todos os testes passaram")
    else:
        print(total_falhas, "testes falharam")

```

```

===== n = 2 =====
add: OK
seed 0 : ok
seed 1 : ok
seed 2 : ok
seed 3 : ok
seed 4 : ok
validador apanhou uma grelha errada: ok
===== n = 3 =====
add: OK
seed 0 : ok
seed 1 : ok
seed 2 : ok
seed 3 : ok
seed 4 : ok
validador apanhou uma grelha errada: ok

```

todos os testes passaram

Para a implementação das funções de validação foi utilizado o recurso à LLM/IA "CLAUDE"

Link para a conversa : <https://claude.ai/share/5e083abb-eac3-442b-87b6-918f548c7586>

Interface

- `seed` : escolhe o puzzle; a mesma seed gera sempre o mesmo.
- `n` : tamanho da grelha ($n^2 \times n^2$). Limitámos a 6 porque para n maiores o tempo de resolução cresce muito (ver "Escala e limites").
- `pistas` : número de pistas, no máximo n^2 , pela razão explicada em `pistas_random`.

```
In [ ]: slider_sudoku = mo.ui.slider(1, 1000, label="seed")
slider_sudoku
```

seed 

```
In [ ]: N_ui = slider_n.value * slider_n.value
slider_pistas = mo.ui.slider(1, N_ui, value=slider_n.value, label="pistas")
slider_pistas
```

pistas 

```
In [ ]: slider_n = mo.ui.slider(2, 6, value=3, label="n")
slider_n
```

n 

```
In [ ]: pistas, grelha = resolver_sudoku(slider_n.value, k=slider_pistas.value,
seed=slider_sudoku.value)
print("Pistas:")
print(mostrar_sudoku(pistas.matriz(), slider_n.value))
print("Solucao:")
print(mostrar_sudoku(grelha, slider_n.value))
```

Pistas:

.	.	.		.	.	.		.	.	4
.	.	.		.	.	.		.	.	5
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
2	.	.		.	.	.		.	.	.

Solucao:

8	3	9		5	7	6		1	2	4
1	2	6		3	4	9		8	7	5
4	7	5		2	1	8		3	6	9
5	6	4		1	2	3		7	9	8
3	1	2		8	9	7		5	4	6
9	8	7		6	5	4		2	1	3
6	5	1		4	3	2		9	8	7
7	4	3		9	8	1		6	5	2
2	9	8		7	6	5		4	3	1

para os sliders acima foi feita uma pesquisa no AI : GEMINI

Link para a conversa : <https://share.google/aimode/9ypkiFvdZDPjGw64B>

Escala e limites

O código não tem nada fixo a 9x9: tudo é calculado a partir de **n** ou de $N = n^2$, por isso funciona para qualquer n sem alterações. Tempo de resolução de um Sudoku normal com $k = n$ pistas:

n	grelha	tempo
2	4x4	< 0,1 s
3	9x9	< 0,1 s
4	16x16	~0,3 s
5	25x25	~1,5 s
6	36x36	~6 s
7	49x49	~12 s
8	64x64	~35 s
9	81x81	> 10 min

Porque cresce tão depressa. Há $N^2 = n^4$ variáveis, cada uma com n^2 valores possíveis, e $3n^2$ restrições "todos diferentes" (linhas, colunas e blocos) com n^2 variáveis cada, mais a do grupo das pistas. Ao passar de $n = 6$ para $n = 9$, o número de variáveis passa de 1296 para 6561 e cada domínio passa de 36 para 81 valores. Além disso, com só $k = n$ pistas a grelha está quase vazia, e o solver tem de construir a solução praticamente do zero.

Limite dos sliders. Como o notebook é reativo, cada mudança num slider volta a correr o solver, e o `solve()` não tem limite de tempo. Limitámos o slider a $n \leq 6$ porque $n = 6$ é o valor sugerido na extensão de escala do enunciado e ainda se resolve em poucos segundos. Para $n = 7$ e $n = 8$ o código funciona (ver tabela), mas o notebook ficava parado dezenas de segundos a cada mudança. Com $n = 9$ ficava bloqueado mais de 10 minutos.

Bónus: Sudoku diagonal (X-Sudoku)

No X-Sudoku as duas diagonais principais também têm de ter valores todos diferentes. Basta acrescentar dois `box` normais: um com as células (i, i) e outro com as células $(i, n^2 - 1 - i)$. A função `montar_sudoku_diagonal` chama a função `montar_sudoku` normal e só acrescenta estes dois grupos. O `Modelo` não mudou nada, o que mostra a vantagem de ter um grupo genérico.

As pistas continuam a poder estar em qualquer célula; só a resolução tem a restrição extra. Com as diagonais, as pistas aleatórias dão mais vezes puzzles impossíveis (sobretudo em 4×4), e a mesma política de 10 tentativas trata disso.

Para validar, `validar_diagonais` aplica o mesmo `valores_corretos` às duas diagonais, e a célula de testes corre o fluxo completo para $n = 2$ e $n = 3$.

```
In [ ]: def montar_sudoku_diagonal(n, pistas):
    N = n * n

    # o sudoku normal (linhas + colunas + blocos + pistas)
    modelo = montar_sudoku(n, pistas)

    # sudoku diagonal: as duas diagonais principais tambem sao grupos
    diagonal1 = box(None, n)
    diagonal2 = box(None, n)
    for i in range(N):
        diagonal1.add(i, i)
        diagonal2.add(i, N - 1 - i)
    modelo.add(diagonal1, diagonal2)

    return modelo

def resolver_sudoku_diagonal(n, k=None, seed=None, tentativas=10):
    # se as pistas derem um puzzle impossivel, tentar outras
    for t in range(tentativas):
```

```

    if seed == None:
        s = None
    else:
        s = seed + t

    pistas = pistas_random(n, k, s)
    grelha = montar_sudoku_diagonal(n, pistas).solve()

    if grelha != None:
        return pistas, grelha

# desistir ao fim de varias tentativas
    return pistas, None

```

```

In [ ]: def validar_diagonais(grelha, n):
    N = n * n
    erros = []

    # juntar os valores das duas diagonais
    diagonal1 = []
    diagonal2 = []
    for i in range(N):
        diagonal1.append(grelha[i][i])
        diagonal2.append(grelha[i][N - 1 - i])

    if not valores_corretos(diagonal1, N):
        erros.append("a diagonal principal está errada")
    if not valores_corretos(diagonal2, N):
        erros.append("a diagonal secundária está errada")

    return erros

```

```

In [ ]: falhas_diagonal = 0

for n_d in [2, 3]:
    print("==== diagonal, n =", n_d, "====")

    for seed_d in range(5):
        pistas_d, grelha_d = resolver_sudoku_diagonal(n_d, seed=seed_d)
        if grelha_d == None:
            print("seed", seed_d, ": sem solucao")
            continue
        erros_d = validar_grelha(grelha_d, n_d) + validar_pistas(grelha_d,
pistas_d) + validar_diagonais(grelha_d, n_d)
        if len(erros_d) == 0:
            print("seed", seed_d, ": OK")
        else:
            print("seed", seed_d, ": FALHOU", erros_d)
            falhas_diagonal = falhas_diagonal + 1

    print()
    if falhas_diagonal == 0:
        print("todos os testes do sudoku diagonal passaram")
    else:
        print(falhas_diagonal, "testes do sudoku diagonal falharam")

```



```

===== diagonal, n = 2 =====
seed 0 : OK
seed 1 : OK
seed 2 : OK
seed 3 : OK
seed 4 : OK
===== diagonal, n = 3 =====
seed 0 : OK
seed 1 : OK
seed 2 : OK
seed 3 : OK
seed 4 : OK

```

todos os testes do sudoku diagonal passaram

```

In [ ]: slider_sudoku_diagonal = mo.ui.slider(1, 10, label="seed diagonal")
        slider_sudoku_diagonal

```

seed diagonal 

```

In [ ]: slider_n_diagonal = mo.ui.slider(2, 6, value=3, label="n")
        slider_n_diagonal

```

n 

```

In [ ]: N_ui_diagonal = slider_n_diagonal.value * slider_n_diagonal.value
        slider_pistas_diagonal = mo.ui.slider(0, N_ui_diagonal,
        value=slider_n_diagonal.value, label="pistas")
        slider_pistas_diagonal

```

pistas 

```

In [ ]: pistas_diagonal, grelha_diagonal =
        resolver_sudoku_diagonal(slider_n_diagonal.value,
        k=slider_pistas_diagonal.value, seed=slider_sudoku_diagonal.value)
        print("Pistas:")
        print(mostrar_sudoku(pistas_diagonal.matriz(), slider_n_diagonal.value))
        print("Solucao:")
        print(mostrar_sudoku(grelha_diagonal, slider_n_diagonal.value))

```

Pistas:

.	.	.		.	.	.		.	.	4
.	.	.		.	.	.		.	.	5
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
.	.	.		.	.	.		.	.	.
2	.	.		.	.	.		.	.	.

Solucao:

8	7	3		2	5	6		9	1	4
1	4	9		3	7	8		2	6	5
5	6	2		1	4	9		8	7	3
4	2	5		6	1	7		3	8	9
3	1	6		8	9	4		5	2	7
9	8	7		5	2	3		1	4	6
6	5	1		4	3	2		7	9	8
7	3	4		9	8	1		6	5	2
2	9	8		7	6	5		4	3	1

Escala e limites (X-Sudoku)

O X-Sudoku usa exatamente o mesmo modelo, apenas com mais 2 restrições "todos diferentes" (as diagonais). Tempo de resolução com $k = n$ pistas:

n	grelha	tempo
2	4x4	< 0,1 s
3	9x9	< 0,1 s
4	16x16	~ 0,3 s
5	25x25	~ 1 s
6	36x36	~ 4 s
7	49x49	~ 10 s
8	64x64	~ 110 s

Comparação com o Sudoku normal. Até $n = 4$ os tempos foram praticamente iguais. De $n = 5$ a $n = 7$ o X-Sudoku foi ligeiramente mais rápido. As restrições extra cortam mais valores logo no início e o solver tem menos possibilidades para explorar. Em $n = 8$ demorou cerca de 3 vezes mais. Isto mostra que o tempo do CP-SAT não depende só do tamanho: depende também de quão "difícil" é a combinação concreta de pistas e

restrições. Mais restrições tanto podem ajudar (menos possibilidades) como atrapalhar (mais difícil encontrar uma solução que cumpra tudo).

Limite dos sliders. Pela mesma razão do Sudoku normal, limitámos o slider a $n \leq 6$.

X-Sudoku: reutilização do Sudoku normal

O X-Sudoku é um Sudoku normal em que as duas diagonais principais também têm de ter valores todos diferentes. Como o `Modelo` aplica `AllDifferent` a qualquer grupo de células, basta acrescentar dois grupos.

Reutilizado sem alterações

- `box`, `cube`, `path`
- `Modelo` (`add` e `solve`)
- `pistas_random`
- `montar_sudoku`
- `mostrar_sudoku`
- `valores_corretos`, `validar_grelha`, `validar_pistas`

O que é novo ou diferente

- `montar_sudoku_diagonal` : chama `montar_sudoku` e junta dois `box` com as diagonais, as células `(i, i)` e `(i, N-1-i)`.
- `resolver_sudoku_diagonal` : igual a `resolver_sudoku`, mas chama `montar_sudoku_diagonal`.
- `validar_diagonais` : nova; aplica `valores_corretos` às duas diagonais.
- **Testes e interface:** o fluxo completo é testado para $n = 2$ e $n = 3$ com 5 seeds cada, como no Sudoku normal, agora também com `validar_diagonais`. O `verificar_add` não se repete porque o `box` é exatamente o mesmo. A interface tem sliders próprios: a seed vai de 1 a 10 e as pistas podem começar em 0.

para a reutilização de código para o X-SUDOKU foi usada a LLM/AI "CLAUDE"

Link para a conversa : <https://claude.ai/share/57666cdf-259d-41fb-ad81-cfa606838ffc>